Eta IR Translation Specification

Thomas Lu (netid:cl2625)

May 8, 2026

Overview

This specification formalizes the translation from a high-level AST of the Eta Programming Language to the required MIR then to the canonical IR. For clarity, we repeat the abstract syntax of the Eta Programming Language (for the .eta files):

$prog$	$::= gd_1\ gd_2\ ...gd_n$	(programs)
gd	$::=$	(global declarations)
	$\mid f(x_1 : t_1, ..., x_n : t_n) : t'_1, ..., t_n\ \{s_1, ..., s_n\}$	
	$\mid x : t$	
	$\mid x : t = e_{\text{lit}}$	
s	$::=$	(statements)
	$\mid \{s_1, ..., s_n\}$	
	$\mid$ if e then s_1	
	$\mid$ if e then s_1 else s_2	
	$\mid$ while e do s	
	$\mid$ return $e_1 ... e_n$	
	$\mid f(e_1, ..., e_n)$	
	$\mid x_1 : td_1, x_2 : td_2, ..., x_n : td_n$	
	$\mid lv_1, lv_2, ..., lv_n = e_1, ..., e_n$	
	$\mid lv_1, lv_2, ..., lv_n = f(e_1, ..., e_m)$	
td	$::=$ int $\mid$ bool $\mid$ arr(td, e) $\mid$ arr(td)	(type specifiers in variable declarations)
lv	$::= _ \mid x \mid x : t \mid e_1[e_2]$	(lvalues)
e	$::=$	(expressions)
	$\mid e_{\text{lit}}$	
	$\mid str$	
	$\mid x$	
	$\mid e_1 \odot e_2$	
	$\mid \square e$	
	$\mid e_1[e_2]$	
	$\mid \{e_1, ..., e_n\}$	
	$\mid$ length(e)	
	$\mid f(e_1, ..., e_n)$	
e_{lit}	$::= n \mid$ true $\mid$ false $\mid char$	(constant literals)
$\odot$	$::=$	(binary expressions)
	$\mid + \mid - \mid * \mid *>> \mid / \mid \% \mid$	
	$\mid == \mid != \mid < \mid \leq \mid > \mid \geq \mid \&\& \mid \mid\mid$	
$\square$	$::= - \mid !$	(unary expressions)

$t ::= \mathsf{int} \mid \mathsf{bool} \mid \mathsf{arr}(t)$ (type specifiers)

We need to translate the above to an intermediate representation with the following abstract syntax and semantics:

$cu ::= dd_1 \ldots dd_n \;\; fd_1 \ldots fd_n$	(IR compilation units)
$dd ::= \mathsf{data}\, L : n_1, \ldots, n_k$	(data declarations)
$fd ::= \mathsf{func}\, L : \sigma$	(function declarations)
$\sigma ::=$	(IR statements)
$\mid d \leftarrow \epsilon$	evaluates location d and then ϵ; assigns evaluated value to location
$\mid \sigma_1 \ldots \sigma_n$	sequential composition of statements
$\mid \mathsf{jump}\, \epsilon$	evaluates ϵ and jumps to the address of ϵ
$\mid \mathsf{cjump}\, \epsilon\; L_1\; L_2$	jumps to L_1 if ϵ evaluates to nonzero and to L_2 otherwise
$\mid L :$	creates label L that points to next address in sequence
$\mid \mathsf{call}\, \epsilon_0(\epsilon_1, \ldots, \epsilon_n)$	evaluates expression left-to-right; puts arguments in temporaries _ARG1, ..., _ARGn, pushes to call stack, and executes function
$\mid \mathsf{return}\, \epsilon_1 \ldots \epsilon_n$	evaluates the expressions left-to-right; stores results in temporaries _RV1, ..., _RVn,then unwinds call stack
$d ::= v \mid [\epsilon]$	(destinations of assignment)
$\epsilon ::=$	(IR expressions)
$\mid n$	integer constant
$\mid t$	temporary variable
$\mid \epsilon_1 \mathsf{op}\, \epsilon_2$	binary expression; evaluates e_1 then e_2
$\mid [\epsilon]$	fetches the content at the address evaluated from e
$\mid \mathsf{call}\, \epsilon_0(\epsilon_1, \ldots, \epsilon_n)$	same as above, but the value of this expression is the first return value
$\mid \&L$	gets the address at label L
$\mid (\sigma;\, \epsilon)$	evaluates statement σ and then expression ϵ
$\mathsf{op} ::=$	(operators)
$\mid \mathsf{add} \mid \mathsf{sub} \mid \mathsf{mul} \mid \mathsf{hmul} \mid \mathsf{div} \mid \mathsf{mod}$	
$\mid \mathsf{and} \mid \mathsf{or} \mid \mathsf{xor} \mid \mathsf{lshift} \mid \mathsf{rshift} \mid \mathsf{arshift}$	
$\mid \mathsf{eq} \mid \mathsf{neq} \mid \mathsf{lt} \mid \mathsf{ult} \mid \mathsf{gt} \mid \mathsf{leq} \mid \mathsf{geq}$	

Translating Expressions

We now describe the translation of Eta expressions. We define the translation function $\mathcal{T}[\![\cdot]\!]_e$ that takes as input an Eta expression and outputs an IR expression. The constant literals are straightforward:

$$\begin{aligned} \mathcal{T}[\![n]\!]_e &= n \\ \mathcal{T}[\![\mathsf{true}]\!]_e &= 1 \\ \mathcal{T}[\![\mathsf{false}]\!]_e &= 0 \\ \mathcal{T}[\![char]\!] &= \mathsf{codepoint_of}(char) \end{aligned}$$

where $\mathsf{codepoint_of}(char)$ is the UTF-8 codepoint of the character.

Accessing identifier x requires different translations depending on whether x is a local or global variable. This is because a global variable will be declared as a data declaration in its compilation

unit and is accessed through a label, while a local variable is accessed through a temporary.

$$\begin{aligned}
\mathcal{T}[\![x]\!]_e &= \mathsf{temp_of}(x) && \text{if } x \text{ is a local variable} \\
\mathcal{T}[\![x]\!]_e &= [\&\mathsf{label_of}(x)] && \text{if } x \text{ is a global variable}
\end{aligned}$$

where $\mathsf{temp_of}(x)$ is the temporary assigned to x and $\mathsf{label_of}(x)$ is the label corresponding to the global variable x. Care must be put into implementing the $\mathsf{temp_of}()$ and $\mathsf{label_of}()$ functions so that naming conflicts don't occur.
Now we consider binary expressions. Those that operate on only integers are easy to translate:

$$\begin{aligned}
\mathcal{T}[\![e_1 - e_2]\!]_e &= \mathcal{T}[\![e_1]\!]_e \ \mathsf{sub}\ \mathcal{T}[\![e_2]\!]_e \\
\mathcal{T}[\![e_1 * e_2]\!]_e &= \mathcal{T}[\![e_1]\!]_e \ \mathsf{mul}\ \mathcal{T}[\![e_2]\!]_e \\
\mathcal{T}[\![e_1 *>> e_2]\!]_e &= \mathcal{T}[\![e_1]\!]_e \ \mathsf{hmul}\ \mathcal{T}[\![e_2]\!]_e \\
\mathcal{T}[\![e_1 / e_2]\!]_e &= \mathcal{T}[\![e_1]\!]_e \ \mathsf{div}\ \mathcal{T}[\![e_2]\!]_e \\
\mathcal{T}[\![e_1 \% e_2]\!]_e &= \mathcal{T}[\![e_1]\!]_e \ \mathsf{mod}\ \mathcal{T}[\![e_2]\!]_e \\
\mathcal{T}[\![e_1 < e_2]\!]_e &= \mathcal{T}[\![e_1]\!]_e \ \mathsf{lt}\ \mathcal{T}[\![e_2]\!]_e \\
\mathcal{T}[\![e_1 > e_2]\!]_e &= \mathcal{T}[\![e_1]\!]_e \ \mathsf{gt}\ \mathcal{T}[\![e_2]\!]_e \\
\mathcal{T}[\![e_1 \leq e_2]\!]_e &= \mathcal{T}[\![e_1]\!]_e \ \mathsf{leq}\ \mathcal{T}[\![e_2]\!]_e \\
\mathcal{T}[\![e_1 \geq e_2]\!]_e &= \mathcal{T}[\![e_1]\!]_e \ \mathsf{geq}\ \mathcal{T}[\![e_2]\!]_e
\end{aligned}$$

However, addition + could operate on both integers and arrays. Hence, we need to incorporate the typing information obtained from semantic analysis to our translation. We only consider integer addition here and delay the discussion of array concatenations to a later part of this section.

$$\mathcal{T}[\![e_1 + e_2]\!]_e = \mathcal{T}[\![e_1]\!] \ \mathsf{add}\ \ \mathcal{T}[\![e_2]\!] \qquad \text{if } e_1 + e_2 \text{ has type } \mathsf{int}$$

Since the == and != operators only test for physical equality on arrays, their translations are also straightforward.

$$\begin{aligned}
\mathcal{T}[\![e_1 == e_2]\!]_e &= \mathcal{T}[\![e_1]\!]_e \ \mathsf{eq}\ \mathcal{T}[\![e_2]\!]_e \\
\mathcal{T}[\![e_1 != e_2]\!]_e &= \mathcal{T}[\![e_1]\!]_e \ \mathsf{neq}\ \mathcal{T}[\![e_2]\!]_e
\end{aligned}$$

For the logical operators, we need to implement short-circuiting as mentioned in lecture.

$$\begin{aligned}
\mathcal{T}[\![e_1 \&\& e_2]\!]_e = (&t \leftarrow 0 \\
&\mathsf{cjump}\ \mathcal{T}[\![e_1]\!]_e\ l_1\ l_f \\
&l_1 : \\
&\mathsf{cjump}\ \mathcal{T}[\![e_2]\!]_e\ l_2\ l_f \\
&l_2 : \\
&t \leftarrow 1 \\
&l_f : \\
&; t) \\
\mathcal{T}[\![e_1 \| e_2]\!]_e = (&t \leftarrow 1 \\
&\mathsf{cjump}\ \mathcal{T}[\![e_1]\!]_e\ l_t\ l_1 \\
&l_1 : \\
&\mathsf{cjump}\ \mathcal{T}[\![e_2]\!]_e\ l_t\ l_2 \\
&l_2 : \\
&t \leftarrow 0 \\
&l_t : \\
&; t)
\end{aligned}$$

Crucially, the labels l_1, l_2, l_f, l_t and temporary t must be fresh for any such translation (e.g. by allocating from a global pool of labels and temporaries). In all subsequent translations, unless

otherwise specified, **all labels and temporaries must be fresh**. We now move on to unary expressions:

$$\begin{aligned}\mathcal{T}[\![-e_1]\!]_e &= 0 \text{ sub } \mathcal{T}[\![e_1]\!] \\ \mathcal{T}[\![!e_1]\!]_e &= 1 \text{ sub } \mathcal{T}[\![e_1]\!]\end{aligned}$$

We now consider the translation of expressions regarding arrays, which are implemented in accordance to the Eta ABI Specification. First, we translate array initializers:

$$\begin{aligned}\mathcal{T}[\![\{e_1, ..., e_n\}]\!]_e = (&t \leftarrow \mathsf{call}\,(\&_\mathsf{eta_alloc})\,(8n+8)) \\ &[t] \leftarrow n \\ &[t \text{ add } 8] \leftarrow \mathcal{T}[\![e_1]\!]_e \\ &[t \text{ add } 16] \leftarrow \mathcal{T}[\![e_2]\!]_e \\ &\quad\vdots \\ &[t \text{ add } 8n] \leftarrow \mathcal{T}[\![e_n]\!]_e \\ &; t+8)\end{aligned}$$

We need to point out two things. First, _eta_alloc is the label of the function reserved by the Eta ABI Specification for byte-sized memory allocation. Second, since the n above is a compile-time constant, so we can directly compute the expressions $8n+8$ and $8n$ during translation (hence the use of algebraic notation instead of abstract syntax of the IR).

Now that we have defined array initializers, realizing that a string literal is stored as an array of its codepoints, we can simply define

$$\mathcal{T}[\![str]\!]_e = \mathcal{T}[\![\{\mathsf{codepoint_of}(str[0]), ..., \mathsf{codepoint_of}(str[\mathsf{len}(str)-1]\}]\!]_e$$

where $\mathsf{len}(str)$ is the length of the string literal and $str[i]$ denotes the ith character of the string literal.

Next, we translate array index expressions and array length expressions.

$$\begin{aligned}\mathcal{T}[\![\mathsf{length}(e_1)]\!]_e &= [\mathcal{T}[\![e_1]\!]_e \text{ sub } 8] \\ \mathcal{T}[\![e_1[e_2]]\!]_e = (&t_a \leftarrow \mathcal{T}[\![e_1]\!] \\ &t_i \leftarrow \mathcal{T}[\![e_2]\!] \\ &\mathsf{cjump}\,(t_i \text{ ult } [t_a \text{ sub } 8])\,l_{\mathsf{ok}}\,l_{\mathsf{err}} \\ &l_{\mathsf{err}}: \\ &\mathsf{call}\,(\&_\mathsf{eta_out_of_bounds})\,() \\ &l_{\mathsf{ok}}: \\ &; [t_a \text{ add } (t_i \text{ mul } 8)])\end{aligned}$$

Note that _eta_out_of_bounds is the name of another function reserved by the ABI to report array indexing errors. Finally, we translate array concatenation expressions. This is a more complex translation where we first evaluate the subexpressions and allocate a new array, and then use two loops to copy the elements of e_1 and e_2 in reverse order using the counter t_n.

$$\begin{aligned}\mathcal{T}[\![e_1 + e_2]\!] = (&t_1 \leftarrow \mathcal{T}[\![e_1]\!]_e && \text{if } e_1 + e_2 \text{ has type } t[] \\ &t_2 \leftarrow \mathcal{T}[\![e_2]\!]_e \\ &t_{1n} \leftarrow [t_1 \text{ sub } 8] \\ &t_{2n} \leftarrow [t_2 \text{ sub } 8] \\ &t_n \leftarrow t_{1n} \text{ add } t_{2n} \\ &t \leftarrow \mathsf{call}\,(\&_\mathsf{eta_alloc})\,((t_n \text{ add } 1) \text{ mul } 8) \\ &[t] \leftarrow t_n \\ &l_2:\end{aligned}$$

$\mathsf{cjump}\,(t_n\ \mathsf{sub}\ t_{1n})\,l_2'\,l_1$
l_2' :
$[t\ \mathsf{add}\ (t_n\ \mathsf{mul}\ 8)] \leftarrow [t_2\ \mathsf{add}\ ((t_n\ \mathsf{sub}\ t_{1n}\ \mathsf{sub}\ 1)\ \mathsf{mul}\ 8)]$
$t_n \leftarrow t_n\ \mathsf{sub}\ 1$
$\mathsf{jump}\ \ \&l_2$
l_1 :
$\mathsf{cjump}\,t_n\,l_1'\,l_0$
l_1' :
$[t\ \mathsf{add}\ (t_n\ \mathsf{mul}\ 8)] \leftarrow [t_1\ \mathsf{add}\ ((t_n\ \mathsf{sub}\ 1)\ \mathsf{mul}\ 8)]$
$t_n \leftarrow t_n\ \mathsf{sub}\ 1$
$\mathsf{jump}\ \ \&l_1$
l_0 :
; $t + 8$)

We finally translate the function call expression:

$$\mathcal{T}[\![f(e_1, ..., e_n)]\!]_e = \mathsf{call}\,(\&\mathsf{label_of}(f))(\mathcal{T}[\![e_1]\!]_e, ..., \mathcal{T}[\![e_n]\!]_e)$$

where $\mathsf{label_of}(f)$ is the label of the function f declared in the compilation unit. We emphasize that $\mathsf{label_of}(f)$ is **not** simply the name of the function and depends on both the name and type of the function in a manner described by the Eta ABI Specification. Therefore, computing the label of f also requires the symbol table, which is an implementation detail that we did not make explicit here.

Translating Statements and Programs

We now describe the translation of Eta Statements. We define the translation function $\mathcal{T}[\![\cdot]\!]_s$ that takes as input an Eta statement and outputs an IR statement. We first consider a block of statements.

$$\mathcal{T}[\![\{s_1, s_2, ..., s_n\}]\!]_s = \mathcal{T}[\![s_1]\!]_s\ \mathcal{T}[\![s_2]\!]_s\ ...\ \mathcal{T}[\![s_n]\!]_s$$

Before we could translate control flow statements, we first need to define a helper translation function $\mathcal{C}[\![e_1, l_1, l_2]\!]$ that takes as input an expression and two labels and outputs a statement that has the effect of jumping to l_1 if e_1 evaluates to true and to L_2 if e_1 evaluates to false.

$$\begin{aligned}
\mathcal{C}[\![\mathsf{true}, l_1, l_2]\!] &= \mathsf{jump}\,\&l_1 \\
\mathcal{C}[\![\mathsf{false}, l_1, l_2]\!] &= \mathsf{jump}\,\&l_2 \\
\mathcal{C}[\![!e_1, l_1, l_2]\!] &= \mathcal{C}[\![e_1, l_2, l_1]\!]
\end{aligned}$$

$$\begin{aligned}
\mathcal{C}[\![e_1 \&\& e_2, l_1, l_2]\!] &= \mathcal{C}[\![e_1, l_t, l_2]\!] \\
&\quad\ l_t : \\
&\quad\ \mathcal{C}[\![e_2, l_1, l_2]\!]
\end{aligned}$$

$$\begin{aligned}
\mathcal{C}[\![e_1 \| e_2, l_1, l_2]\!] &= \mathcal{C}[\![e_1, l_1, l_t]\!] \\
&\quad\ l_t : \\
&\quad\ \mathcal{C}[\![e_2, l_1, l_2]\!]
\end{aligned}$$

$$\mathcal{C}[\![e_1, l_1, l_2]\!] = \mathsf{cjump}\ \mathcal{T}[\![e_1]\!]_e\,l_1\,l_2 \qquad \text{(if none of the above rules apply)}$$

We now translate control flow statements.

$$\mathcal{T}[\![\mathsf{if}\,e_1\ \ \mathsf{then}\ s_1]\!]_s = \mathcal{C}[\![e_1, l_1, l_2]\!]$$

$$
\begin{aligned}
& l_1 : \\
& \mathcal{T}[\![s_1]\!]_s \\
& l_2 :
\end{aligned}
$$

$$
\begin{aligned}
\mathcal{T}[\![\mathsf{if}\, e_1 \ \mathsf{then}\ s_1 \ \mathsf{else}\ s_2]\!]_s = \ & \mathcal{C}[\![e_1, l_1, l_2]\!] \\
& l_1 : \\
& \mathcal{T}[\![s_1]\!]_s \\
& \mathsf{jump}\, \&l_3 \\
& l_2 : \\
& \mathcal{T}[\![s_2]\!]_s \\
& l_3 :
\end{aligned}
$$

$$
\begin{aligned}
\mathcal{T}[\![\mathsf{while}\ e_1\ \mathsf{do}\ s_1]\!]_s = \ & l_{\mathsf{start}} : \\
& \mathcal{C}[\![e_1, l_{\mathsf{body}}, l_{\mathsf{end}}]\!] \\
& l_{\mathsf{body}} : \\
& \mathcal{T}[\![s_1]\!]_s \\
& \mathsf{jump}\, \&l_{\mathsf{start}} \\
& l_{\mathsf{end}} :
\end{aligned}
$$

$$
\mathcal{T}[\![\mathsf{return}\ e_1 \ldots e_n]\!]_s = \mathsf{return}\ \ \mathcal{T}[\![e_1]\!]_e \ldots \mathcal{T}[\![e_n]\!]_e
$$

The translation of declarations is more complex than it seems. First, we need to define a recursive function $\mathcal{A}[\![t_1, ..., t_n]\!]$ that takes in a list of temporaries. Intuitively, $\mathcal{A}[\![\cdot]\!]$ returns an IR expression that allocates an n-dimensional array whose dimension is $t_1 \times t_2 \times ... \times t_n$.

$$
\begin{aligned}
\mathcal{A}[\![t_i :: \ell]\!] = \ & (t_{\mathsf{counter}} \leftarrow t_i \\
& t \leftarrow \mathsf{call}\, (\&_\mathsf{eta_alloc})\, ((t_i\ \mathsf{add}\ 1)\ \mathsf{mul}\ 8) \\
& [t] \leftarrow t_i \\
& l_{\mathsf{start}} : \\
& \mathsf{cjump}\, t_{\mathsf{counter}}\, l_{\mathsf{body}}\, l_{\mathsf{end}} \\
& l_{\mathsf{body}} : \\
& [t\ \mathsf{add}\ (t_{\mathsf{counter}}\ \mathsf{mul}\ 8)] \leftarrow \mathcal{A}[\![\ell]\!] \\
& t_{\mathsf{counter}} \leftarrow t_{\mathsf{counter}}\ \mathsf{add}\ (-1) \\
& \mathsf{jump}\, \&l_{\mathsf{start}} \\
& l_{\mathsf{end}} : \\
& ; t + 8) \\
\mathcal{A}[\![[]]\!] = \ & 0
\end{aligned}
$$

We could then translate a declaration by adding the following definitions to the translation function $\mathcal{T}[\![\cdot]\!]_s$:

$$
\begin{aligned}
\mathcal{T}[\![x_i : \mathsf{int}]\!] = \ & \mathsf{temp_of}(x_i) \leftarrow 0 \\
\mathcal{T}[\![x_i : \mathsf{bool}]\!] = \ & \mathsf{temp_of}(x_i) \leftarrow 0 \\
\mathcal{T}[\![x_i : \mathsf{arr}(\mathsf{arr}(...\mathsf{arr}(td, e_n)..., e_2), e_1)]\!] = \ & t_1 \leftarrow \mathcal{T}[\![e_1]\!]_e \\
& \vdots \\
& t_n \leftarrow \mathcal{T}[\![e_n]\!]_e \\
& \mathsf{temp_of}(x_i) \leftarrow \mathcal{A}[\![t_1, ..., t_n]\!] \\
\mathcal{T}[\![x_1 : td_1, ..., x_n : td_n]\!]_s = \ & \mathcal{T}[\![x_1 : td_1]\!]_s \ldots \mathcal{T}[\![x_n : td_n]\!]_s
\end{aligned}
$$

We use a similar strategy to handle assignments. That is, we first define a helper translation function $\mathcal{T}[\![\cdot]\!]_{lv}$ that takes an input an lvalue and outputs a destination (which can be either a temporary or a memory access $[e]$) to which the lvalue's corresponding expression is to be assigned.

$$
\begin{aligned}
\mathcal{T}[\![_]\!]_{lv} &= \mathsf{wildcard} \\
\mathcal{T}[\![x]\!]_{lv} &= \mathsf{temp_of}(x) \\
\mathcal{T}[\![x : t]\!]_{lv} &= \mathsf{temp_of}(x) \\
\mathcal{T}[\![e_1[e_2]]\!]_{lv} &= [\\
&\quad (t_a \leftarrow \mathcal{T}[\![e_1]\!]_e \\
&\quad t_i \leftarrow \mathcal{T}[\![e_2]\!]_e \\
&\quad \mathsf{cjump}\,(t_i\ \mathsf{ult}\ [t_a\ \mathsf{sub}\ 8])\,l_{\mathsf{ok}}\,l_{\mathsf{err}} \\
&\quad l_{\mathsf{err}} : \\
&\quad \mathsf{call}\,(\&_\mathsf{eta_out_of_bounds})\,() \\
&\quad l_{\mathsf{ok}} : \\
&\quad ;\ t_a\ \mathsf{add}\ (t_i\ \mathsf{mul}\ 8)) \\
&\quad]
\end{aligned}
$$

We then translate assignments as:

$$
\begin{aligned}
\mathcal{T}[\![lv_1, ..., lv_n = e_1, ..., e_n]\!]_s &= t_1 \leftarrow \mathcal{T}[\![e_1]\!]_e \\
&\quad \vdots \\
&\quad t_n \leftarrow \mathcal{T}[\![e_n]\!]_e \\
&\quad \mathcal{T}[\![lv_1]\!]_{lv} \leftarrow t_1 \\
&\quad \vdots \\
&\quad \mathcal{T}[\![lv_n]\!]_{lv} \leftarrow t_n
\end{aligned}
$$

$$
\begin{aligned}
\mathcal{T}[\![lv_1, ..., lv_n = f(e_1, ..., e_m)]\!]_s &= \mathsf{call}\,(\&\mathsf{label_of}(f))\,(\mathcal{T}[\![e_1]\!]_e, ..., \mathcal{T}[\![e_m]\!]_e) \\
&\quad \mathcal{T}[\![lv_1]\!]_{lv} \leftarrow _\mathsf{RV1} \\
&\quad \vdots \\
&\quad \mathcal{T}[\![lv_n]\!]_{lv} \leftarrow _\mathsf{RV}n
\end{aligned}
$$

A procedural call can again be simply translated as

$$
\mathcal{T}[\![f(e_1, ..., e_n)]\!]_s = \mathsf{call}\,(\&\mathsf{label_of}(f))\,(\mathcal{T}[\![e_1]\!]_e, ..., \mathcal{T}[\![e_m]\!]_e)
$$

Finally, we can translate global declarations by defining the $\mathcal{T}[\![\cdot]\!]_{gd}$ function.

$$
\begin{aligned}
\mathcal{T}[\![f(x_1 : t_1, ..., x_n : t_n) : t'_1, ..., t_n\ \{s_1, ..., s_n\}]\!]_{gd} &= \mathsf{func\ label_of}(f) : \\
&\quad \mathsf{temp_of}(x_1) \leftarrow _\mathsf{ARG1} \\
&\quad \vdots \\
&\quad \mathsf{temp_of}(x_n) \leftarrow _\mathsf{ARG}n \\
&\quad \mathcal{T}[\![\{s_1, ..., s_n\}]\!]_s
\end{aligned}
$$

$$
\begin{aligned}
\mathcal{T}[\![x : t]\!]_{gd} &= \mathsf{data\ label_of}(x) : 0 \\
\mathcal{T}[\![x : t = e_{\mathsf{lit}}]\!]_{gd} &= \mathsf{data\ label_of}(x) : \mathcal{T}[\![e_{\mathsf{lit}}]\!]_e
\end{aligned}
$$

To translate a program, we first need to reorder the global declarations inside a program so that all data declarations come before function declarations. After this preprcessing step, we can simply

translate

$$\mathcal{T}[\![gd_1\ gd_2 \ldots gd_n]\!]_{prog} = \mathcal{T}[\![gd_1]\!]_{gd}\ \mathcal{T}[\![gd_2]\!]_{gd} \ldots \mathcal{T}[\![gd_n]\!]_{gd}$$

(Premature) Optimizations

This section describes certain local optimizations that needs to be done at the IR level to facilitate effective instruction selection at the assembly level. Starting from the more minor optimization, we can rewrite the translation of unary negations such that

$$\mathcal{T}[\![!e_1]\!]_e = \mathcal{T}[\![e_1]\!]\ \mathsf{xor}\ 1$$

The more crucial problem hindering instruction selection is that the current lowering transformation applied to IR expressions is too conservative. Recall that IR expressions are lowered such that

$$\frac{\mathcal{L}[\![e_1]\!] = \overrightarrow{s_1}; e_1' \qquad \mathcal{L}[\![e_2]\!] = \overrightarrow{s_2}; e_2'}{\mathcal{L}[\![e_1\ \mathsf{op}\ e_2]\!] = \overrightarrow{s_1}\ (t_1 \leftarrow e_1')\ \overrightarrow{s_2}; t_1\ \mathsf{op}\ e_2'}\ \textsc{BinOpDumb}$$

However, if we can show that evaluating $\overrightarrow{s_2}$ does not affect the evaluation of e_1, denoted as $\overrightarrow{s_2} \not\rightarrow e_1$, then we can apply the following version of the translation

$$\frac{\mathcal{L}[\![e_1]\!] = \overrightarrow{s_1}; e_1' \qquad \mathcal{L}[\![e_2]\!] = \overrightarrow{s_2}; e_2' \qquad \overrightarrow{s_2} \not\rightarrow e_1}{\mathcal{L}[\![e_1\ \mathsf{op}\ e_2]\!] = \overrightarrow{s_1}\overrightarrow{s_2}; e_1'\ \mathsf{op}\ e_2'}\ \textsc{BinOpSmart}$$

In the absence of a full dataflow analyzer, we need a local, conservative estimation of $\overrightarrow{s_2} \not\rightarrow e_1$. More formally, we say $\overrightarrow{s_2} \not\rightarrow e_1$ if and only if $\overrightarrow{s_2}$ does not contain IR statements of the following form:

- a memory assignment $[\epsilon'] \leftarrow \epsilon''$
- a function call statement $\mathsf{call}\ \epsilon_0(\epsilon_1, ..., \epsilon_n)$
- an assignment to temporary $t \leftarrow \epsilon'$ such that t is not in the set of temporaries that e_1 uses

,
With the above definition we can also use the following smarter translation of memory writes:

$$\frac{\mathcal{L}[\![e_1]\!] = \overrightarrow{s_1}; e_1' \qquad \mathcal{L}[\![e_2]\!] = \overrightarrow{s_2}; e_2' \qquad \overrightarrow{s_2} \not\rightarrow e_1}{\mathcal{L}[\![[e_1] \leftarrow e_2]\!] = \overrightarrow{s_1}\overrightarrow{s_2}; [e_1'] \leftarrow e_2'}\ \textsc{MemWriteSmart}$$

In addition, we need to modify certain translations that involve arithmetic expressions so that they are in a normal form that can be detected during instruction selection. Ideally, equality saturation would be able to transform arithmetic expressions into normal forms automatically, but currently we must make do without such automation. In the following passage, we document the translation that needs to be changed. The specific arithmetic expressions that are changed are wrapped in boxes.

$$\begin{aligned}
\mathcal{T}[\![\mathsf{length}(e_1)]\!]_e &= [\boxed{\mathcal{T}[\![e_1]\!]_e\ \mathsf{add}\ (-8)}] \\
\mathcal{T}[\![e_1[e_2]]\!]_e &= (t_a \leftarrow \mathcal{T}[\![e_1]\!] \\
&\quad\ t_i \leftarrow \mathcal{T}[\![e_2]\!] \\
&\quad\ \mathsf{cjump}\,(t_i\ \mathsf{ult}\ [\boxed{t_a\ \mathsf{add}\ (-8)}])\ l_{\mathsf{ok}}\ l_{\mathsf{err}} \\
&\quad\ l_{\mathsf{err}}: \\
&\quad\ \mathsf{call}\,(\&_\mathsf{eta_out_of_bounds})\,() \\
&\quad\ l_{\mathsf{ok}}: \\
&\quad\ ;\ [t_a\ \mathsf{add}\ (t_i\ \mathsf{mul}\ 8)])
\end{aligned}$$

$\mathcal{T}[\![e_1 + e_2]\!] = (t_1 \leftarrow \mathcal{T}[\![e_1]\!]_e$ if $e_1 + e_2$ has type $t\,[\,]$

$t_2 \leftarrow \mathcal{T}[\![e_2]\!]_e$

$t_{1n} \leftarrow [\boxed{t_1 \text{ add } (-8)}]$

$t_{2n} \leftarrow [\boxed{t_2 \text{ add } (-8)}]$

$t_n \leftarrow t_{1n} \text{ add } t_{2n}$

$t \leftarrow \text{call } (\&_\text{eta_alloc}) (\boxed{(t_n \text{ mul } 8) \text{ add } 8})$

$[t] \leftarrow t_n$

$l_2 :$

$\text{cjump } (t_n \text{ sub } t_{1n})\, l_2'\, l_1$

$l_2' :$

$[t \text{ add } (t_n \text{ mul } 8)] \leftarrow [\boxed{t_2 \text{ add } ((t_n \text{ sub } t_{1n}) \text{ mul } 8) \text{ add } (-8)}]$

$t_n \leftarrow \boxed{t_n \text{ add } (-1)}$

$\text{jump } \&l_2$

$l_1 :$

$\text{cjump } t_n\, l_1'\, l_0$

$l_1' :$

$[t \text{ add } (t_n \text{ mul } 8)] \leftarrow [\boxed{t_1 \text{ add } (t_n \text{ mul } 8) \text{ add } (-8)}]$

$t_n \leftarrow \boxed{t_n \text{ add } (-1)}$

$\text{jump } \&l_1$

$l_0 :$

$; t + 8)$

Extending the translation to Rho

In the Rho extension of the Eta Programming Language, the structure of the AST has been extended to

$prog ::= gd_1\, gd_2 \ldots gd_n$	(programs)
$gd ::=$	(global declarations)
$\mid f(x_1 : t_1, \ldots, x_n : t_n) : t_1', \ldots, t_n \{s_1, \ldots, s_n\}$	
$\mid x : t$	
$\mid x : t = e_{\text{lit}}$	
$s ::=$	(statements)
$\mid \{s_1, \ldots, s_n\}$	
$\mid \text{if } e \text{ then } s_1$	
$\mid \text{if } e \text{ then } s_1 \text{ else } s_2$	
$\mid \text{while } e \text{ do } s$	
$\mid \boxed{\text{break}}$	
$\mid \text{return } e_1 \ldots e_n$	
$\mid f(e_1, \ldots, e_n)$	
$\mid x_1 : td_1, x_2 : td_2, \ldots, x_n : td_n$	
$\mid lv_1, lv_2, \ldots, lv_n = e_1, \ldots, e_n$	
$\mid lv_1, lv_2, \ldots, lv_n = f(e_1, \ldots, e_m)$	
$td ::= \text{int} \mid \text{bool} \mid \text{arr}(td, e) \mid \text{arr}(td)$	(type specifiers in variable declarations)
$\mid \boxed{\text{record } r}$	

$$
\begin{array}{rll}
lv ::= & _ \mid x \mid x : t \mid e_1[e_2] & \text{(lvalues)} \\
& \mid \boxed{e.x} & \\
\\
e ::= & & \text{(expressions)} \\
& \mid e_{\mathsf{lit}} & \\
& \mid str & \\
& \mid x & \\
& \mid e_1 \odot e_2 & \\
& \mid \square e & \\
& \mid e_1[e_2] & \\
& \mid \{e_1, ..., e_n\} & \\
& \mid \mathsf{length}(e) & \\
& \mid f(e_1, ..., e_n) & \\
& \mid \boxed{e.x} & \\
& \mid \boxed{\mathsf{null}} & \\
e_{\mathsf{lit}} ::= & n \mid \mathsf{true} \mid \mathsf{false} \mid char & \text{(constant literals)} \\
\odot ::= & & \text{(binary expressions)} \\
& \mid + \mid - \mid * \mid *>> \mid / \mid \% \mid & \\
& \mid == \mid != \mid < \mid \leq \mid > \mid \geq \mid \&\& \mid || & \\
\square ::= & - \mid ! & \text{(unary expressions)} \\
t ::= & \mathsf{int} \mid \mathsf{bool} \mid \mathsf{arr}(t) & \text{(type specifiers)} \\
& \mid \boxed{\mathsf{record}\ r} &
\end{array}
$$

We first handle the translation of break statements. Define:

$$\mathcal{T}[\![\mathsf{break}]\!]_s = \mathsf{jump}\ \&L_{\mathsf{cont}}$$

where L_{cont} the l_{end} label generated by the translation of the smallest while loop containing the break statements. We can implement this as an attribute in the visitor that gets updated whenever an while loop is encountered.

Now we define the translation for declaring a record:

$$
\begin{array}{rll}
\mathcal{T}[\![x : \mathsf{record}\, r]\!]_s = & \mathsf{temp_of}(x) \leftarrow \mathsf{call}\ (\&_\mathsf{eta_alloc})\ (8n) & \text{if } n \geq 1 \\
& [\mathsf{temp_of}(x)] \leftarrow 0 & \\
& [\mathsf{temp_of}(x)\ \mathsf{add}\ 8] \leftarrow 0 & \\
& \vdots & \\
& [\mathsf{temp_of}(x)\ \mathsf{add}\ 8(n-1)] \leftarrow 0 & \\
\\
\mathcal{T}[\![x : \mathsf{record}\, r]\!]_s = & \mathsf{temp_of}(x) \leftarrow 0 & \text{if } n = 0
\end{array}
$$

where n is the length of the record type $\mathsf{record}\, r$ in the current symbol table containing the declaration. Since record fields can be accessed as lvalues, we also need to amend the translation for lvalues. We assume in the following translation, the conditions $\Gamma \vdash e : \mathsf{record}\, r$ and $\Gamma(r) = \mathsf{record}\, \{x_1 : t_1, ..., x_n : t_n\}$ hold where lvalue is located.

$$\mathcal{T}[\![e.f]\!]_{lv} = [\mathcal{T}[\![e]\!]_e\ \mathsf{add}\ (i-1)]$$

where i is such that $f = f_i$. Finally, we translate the two newly added forms of expressions below:

$$
\begin{aligned}
\mathcal{T}[\![e.f]\!]_e &= [\mathcal{T}[\![e]\!]_e\ \mathsf{add}\ (i-1)] \\
\mathcal{T}[\![\mathsf{null}]\!] &= 0
\end{aligned}
$$